

MODULE 31 · EXAM STRATEGY - ALL DOMAINS

Confusion pairs

Forty things that look alike, and the one detail that separates each pair.

COVERS All 7 domains TYPE Revision MODULE 31 of 36

For each pair, do not memorise both.
Memorise **the one detail that separates them**, and you can derive the rest.

Compute and platform

THE PAIR	THE DISCRIMINATOR
Jobs compute / all-purpose	Terminates with the job / bills 2–3x more
Standard / dedicated access mode	Dedicated for Runtime ML, GPU, RDD API and R
Classic / serverless SQL warehouse	Cold start: about 4 minutes / seconds
Control plane / compute plane	Notebooks and job config / processes data
Managed / external table	DROP deletes the files / leaves them

ON THE EXAM

The last row is the most consequential distinction in the course.

Ingestion

THE PAIR	THE DISCRIMINATOR
COPY INTO / Auto Loader	Thousands of files is COPY INTO. Millions is Auto Loader.
mergeSchema: FORMAT_OPTIONS / COPY_OPTIONS	Reconciles source files / evolves the target table
FILES / PATTERN	Mutually exclusive. FILES caps at 1000.
addNewColumns / rescue	Fails then resumes / never fails, never evolves
failOnNewColumns / none	Fails and will not restart / ignores the column
Directory listing / file events	File events is recommended, and must run at least every 7 days.

ON THE EXAM

Auto Loader's four evolution modes are best learnt as two pairs.

Writing data

THE PAIR	THE DISCRIMINATOR
saveAsTable / insertInto	By name / by position. insertInto swaps silently.
CREATE OR REPLACE TABLE / INSERT OVERWRITE	DDL replaces the schema / DML keeps it
append / overwrite / error / ignore	error is the default, not append
cast / try_cast	Under ANSI cast throws. try_cast returns null.
MERGE / append	MERGE is idempotent on a key. append is not.

ON THE EXAM

Row one is a silent failure: same-type columns raise no error.

Combining and deduplicating

THE PAIR	THE DISCRIMINATOR
SQL UNION / DataFrame union()	Deduplicates / does not. Nor does unionAll.
union() / unionByName()	By position / by name. Same hazard as insertInto.
distinct() / dropDuplicates(subset)	Whole rows / named columns
dropDuplicates(subset) / row_number window	Arbitrary row / you state which one wins
EXCEPT / left_anti	Whole rows / join keys only
explode / explode_outer	Drops empty arrays / keeps them

ON THE EXAM

Rows one and two are where SQL habits mislead you most.

Aggregation and joins

THE PAIR	THE DISCRIMINATOR
<code>count(*) / count(col)</code>	Rows / non-null values. They disagree exactly when nulls exist.
<code>count_distinct / approx_count_distinct</code>	Exact / estimated. <code>rsd</code> defaults to 0.05; below 0.01 use the exact one.
<code>describe() / summary()</code>	<code>summary</code> adds quartiles, which is what reveals skew.
Filter before <code>groupBy</code> / after	Before changes the values. After changes which groups appear.
<code>groupBy</code> / window function	<code>groupBy</code> collapses rows. A window keeps them and adds the aggregate.
Broadcast / sort-merge join	Move the join to the data / move the data to the join.

ON THE EXAM

Filter placement is the most commonly tested of these, because it changes both the values and the row count.

Gold objects and quality

THE PAIR	THE DISCRIMINATOR
View / materialised view	A view is always current and always costs. An MV is fast and can be stale.
Materialised view / streaming table	If the source must be read only once, it must be a streaming table.
Table constraint / pipeline expectation	A constraint binds every writer. An expectation binds the pipeline.
warn / drop / fail	warn is the default, and it writes the invalid record to the target.
drop / quarantine	drop keeps only a count. Quarantine keeps the rows for review.
fail / the others	fail records no metrics, because the update stops on detection.

ON THE EXAM

warn being the default is the single most testable fact in Domain 3's quality objective.

Jobs

THE PAIR	THE DISCRIMINATOR
Run if / If/else	Run if branches on outcome. If/else branches on a value.
Upstream failed / Excluded	Excluded counts as successful. Upstream failed does not.
Succeeded / Succeeded with failures	Job status comes from leaf tasks. A Failed filter hides the second.
Retry / repair	A retry re-runs a task. A repair re-runs a run's unsuccessful tasks.
Cooldown / debouncing	Cooldown limits frequency. Debouncing waits for the batch to finish arriving.
Triggered / continuous pipeline	Only a triggered pipeline can be a job task.

ON THE EXAM

Rows two and three combine into Domain 4's most surprising behaviour: a job can report success while a task inside it failed.

CI/CD

THE PAIR	THE DISCRIMINATOR
Git folders / bundles	A development tool / a deployment mechanism.
Development mode / production mode	Development prefixes resource names. Production validates the Git branch.
OAuth U2M / M2M	A person at a keyboard / a service principal. A CI runner is nobody.
PAT / OAuth	PATs are long-lived and now labelled legacy.
validate / deploy	validate touches no workspace. It belongs on every pull request.
Source format / .ipynb	Source gives clean diffs. IPYNB preserves dashboards and visualisations.

ON THE EXAM

The first row decides most of Domain 5. If a question mentions dev, test and prod differing, you have left Git folders behind.

Optimisation

THE PAIR

Skew / under-resourced

Spill (memory) / spill (disk)

Driver OOM / executor OOM

shuffle.partitions / default.parallelism

Partitioning / liquid clustering

Z-ORDER / liquid clustering

THE DISCRIMINATOR

A few slow tasks / all tasks slow. Opposite fixes.

Deserialised / serialised. Memory is normally much larger.

collect, toPandas, broadcast / partition size and skew.

Spark SQL / RDDs only.

Partition columns are permanent. Clustering keys can change.

Predictive optimisation ignores Z-ordered files.

ON THE EXAM

The first row is the discriminator the exam's own sample question is built on. Max versus the 75th percentile.

Governance

THE PAIR	THE DISCRIMINATOR
SELECT / USE CATALOG and USE SCHEMA	SELECT is not enough. Traversal is a separate privilege.
Ownership / MANAGE	MANAGE grants the ability to grant. It is not all privileges.
ALL PRIVILEGES / actually all	It excludes MANAGE and the EXTERNAL USE privileges.
Row filter / column mask	Returns boolean / returns the column's own type.
Dynamic view / table-level policy	The view protects one object. The policy follows the table.
ABAC / table-level	Catalog or schema level, by tag / one table, by ALTER TABLE.
WHEN / MATCH COLUMNS	WHEN matches securables. MATCH COLUMNS matches columns.

ON THE EXAM

Row one is the most common real access failure in Unity Catalog, and row three is the most quietly misleading name.

warn is the default. union does not deduplicate. dropDuplicates picks arbitrarily. **A few slow tasks is skew; all slow is not.** Column tags do not inherit.

Three shapes of wrong answer

- ◆ **The other member of the pair.** Plausible, related, and wrong on one specific detail.
- ◆ **An invented limit or rule.** 'Not supported above 10 nodes', 'requires a minimum value'.
- ◆ **A true statement that does not answer the question.** Correct, and irrelevant to what was asked.
- ◆ **The correct answer often names a mechanism,** rather than describing a symptom or restating the problem.

ON THE EXAM

You have met all three shapes repeatedly in this course's scenarios. Recognising the shape narrows four options to two.

A pipeline writes a DataFrame with columns (order_id, customer_ref, region_code) into an existing table whose columns are (order_id, region_code, customer_ref). All three are strings. The write succeeds. No error is raised. Weeks later, reporting shows region codes appearing as customer references.

Which write method was used, and what should have been used?

- ☐ A saveAsTable was used; insertInto would have matched by name
- ☐ B insertInto was used; saveAsTable matches by name and would have aligned the columns
- ☐ C Either would produce this; the schemas must be reordered at source
- ☐ D append mode was used; overwrite would have corrected the ordering

◆ PAUSE AND DECIDE

Which write method was used, and what should have been used?

- A

`saveAsTable` used; `insertInto` matches by name
Exactly backwards. `saveAsTable` matches by name; `insertInto` matches by position.
- B

`insertInto` used; `saveAsTable` matches by name
Correct. Positional matching put the second column into the second slot, and all three being strings meant no type error surfaced.
- C

 Either would do this
`saveAsTable` would have aligned the columns by name and written correctly. The methods genuinely differ here.
- D

`append` versus `overwrite`
Write mode governs what happens to existing rows, not how columns are matched.

THE SIGNAL

'All three are strings' is doing the work. A type mismatch would have raised an error and the bug would have been caught immediately.

An engineer must produce a list of every customer who has never placed an order. She writes a query using EXCEPT between the customers table and the orders table, and it returns every customer.

Both tables are in Unity Catalog and are correctly populated.

Why did EXCEPT return everything?

- ☐ A EXCEPT compares whole rows, and the two tables have different schemas, so no row matches
- ☐ B EXCEPT requires both tables to be sorted first
- ☐ C EXCEPT deduplicates, which removed the matching rows
- ☐ D She should have used INTERSECT

◆ PAUSE AND DECIDE

Why did EXCEPT return everything?

- A

 EXCEPT compares whole rows
Correct. A customer row and an order row are never identical, so nothing is excluded. She needs left_anti on customer_id.
- B

 It requires sorting
Invented rule. Set operations in Spark do not depend on ordering.
- C

 Deduplication removed them
EXCEPT does deduplicate, and that affects duplicates within the result, not which rows are excluded.
- D

 Use INTERSECT
INTERSECT returns rows in both, which is the opposite question, and it would return nothing for the same reason.

THE SIGNAL

EXCEPT compares whole rows; left_anti compares join keys. This question needs a key comparison, so it needs the anti join.

Two jobs are investigated on the same morning. Job A's longest stage: median task 14 seconds, 75th percentile 19 seconds, max 26 minutes. Job B's longest stage: median 11 minutes, 75th percentile 12 minutes, max 12.5 minutes.

Neither reports any spill.

What do they need?

- ☐ A Both need more shuffle partitions
- ☐ B A needs skew handling; B needs more capacity or parallelism
- ☐ C A needs more memory; B needs skew handling
- ☐ D Both need more executor memory

◆ PAUSE AND DECIDE

What do they need?

A Both need more shuffle partitions
Would help B. It does not split A's single hot key, which is one partition doing nearly all the work.

C A needs memory; B needs skew
Inverts both diagnoses. Neither reports spill, and B's distribution shows no skew at all.

B A is skew; B is capacity
Correct. A's max is vastly above its 75th percentile. B's is 4% above, which is about as even as a distribution gets.

D Both need more memory
Neither reports spill, which is the symptom memory pressure produces. Adding memory addresses a problem neither has.

THE SIGNAL

Same tab, same three numbers, opposite conclusions. Compare Max to the 75th percentile: 26 minutes against 19 seconds, or 12.5 against 12.

A team needs a gold object holding daily totals, computed from a silver table that is rebuilt nightly with a full overwrite. Queries against it must always reflect the most recent nightly build, and roughly 200 analysts query it each morning. The computation takes six minutes. A colleague suggests a streaming table so it stays continuously up to date.

What should they build?

- ☐ A A streaming table, as suggested
- ☐ B A materialised view, refreshed after the nightly build
- ☐ C A plain view, so it is always current
- ☐ D A dynamic view

◆ PAUSE AND DECIDE

What should they build?

A A streaming table

Streaming tables are append-only and process each record once. The source is fully overwritten nightly, which they cannot consume.

C A plain view

Always current, and 200 analysts each pay six minutes. The data only changes once a day, so 199 of those are waste.

B A materialised view refreshed after the build

Correct. Compute once at six minutes; 200 analysts read a stored result. Freshness matches the nightly build exactly.

D A dynamic view

Dynamic views vary output by user identity. Nothing here differs per user.

THE SIGNAL

A full overwrite each night is the detail that eliminates the streaming table. Append-only cannot consume a source that is replaced.

A job must skip an expensive rebuild on days when its upstream ingestion task loaded no new rows. The ingestion task succeeds on those days, having simply found nothing to load. An engineer configures the rebuild task with Run if set to 'At least one succeeded'.

Why will this not work?

- ☐ A Run if branches on outcome, and the ingestion task succeeds either way; this needs If/else on a task value
- ☐ B 'At least one succeeded' should be 'All succeeded'
- ☐ C The rebuild task should be disabled on those days
- ☐ D Run if conditions cannot be applied to tasks with a single dependency

◆ PAUSE AND DECIDE

Why will this not work?

- A

Run if sees outcome; this needs If/else on a value

Correct. Loading zero rows is a success, so no Run if condition can distinguish the two days. Only the row count can.
- B

Use 'All succeeded' instead

Both conditions are satisfied on both kinds of day, because the ingestion task succeeds either way. Swapping them changes nothing.
- C

Disable the task

Disabling is a manual configuration change, so somebody must check for data and edit the job each morning.
- D

Run if needs multiple dependencies

Invented restriction. Run if applies regardless of how many dependencies a task has.

THE SIGNAL

Nothing about the outcome differs between the two days. When outcome cannot distinguish them, Run if cannot help.

A governance team tags a catalog with sensitivity:high, then writes an ABAC column mask policy on that catalog using MATCH COLUMNS has_tag_value('pii','ssn').

Nothing is masked. Separately, a colleague reports that a user with SELECT on a table in that catalog cannot query it at all.

What are the two causes?

- ☐ A Both are the same cause: the catalog tag has not propagated
- ☐ B The columns were never tagged pii:ssn — column tags do not inherit; and the user is missing USE CATALOG or USE SCHEMA
- ☐ C has_tag_value is unsupported; and the user needs ALL PRIVILEGES
- ☐ D The policy must be attached to each table; and the user needs MANAGE

◆ PAUSE AND DECIDE

What are the two causes?

- A

 One cause: propagation
Two unrelated problems. And the catalog tag did propagate – it is simply not the tag the policy matches on.
- B

 Column tags do not inherit; and traversal is missing
Correct. MATCH COLUMNS needs column-level tags applied directly, and reaching a table needs USE CATALOG and USE SCHEMA as well as SELECT.
- C

 has_tag_value unsupported; needs ALL PRIVILEGES
has_tag_value is one of only two supported functions. And ALL PRIVILEGES is far more than traversal requires.
- D

 Attach per table; user needs MANAGE
Catalog-level attachment is correct and intended. MANAGE grants the ability to grant, which is not what reading a table needs.

THE SIGNAL

Two Domain 7 pairs in one scenario: tag inheritance stops at columns, and SELECT is not traversal.

When two options both look right, you have found a pair. **Ask which single detail separates them**, and the question usually answers itself.